

Supplementary Report

1 Consumer Browsing Data Collection and Transformation

As shown in Figure 1, we deployed a UWB positioning system based on the Qorvo DW3000 module inside an electronics retail store measuring $64m$ in length, $38m$ in width, and $4.2m$ in height, with a total floor area of approximately $2,500m^2$. The system comprises 46 ceiling-mounted anchors and 300 mobile tags, of which 200 were attached to shopping carts and 100 to shopping baskets, covering the majority of individual shoppers. Positioning anchors are installed above each product zone within the store, as illustrated in Figure 2(a). To avoid interfering with the consumer shopping experience, UWB positioning tags are discreetly attached to the bottoms of shopping baskets and carts, as shown in Figures 2(b) and 2(c). Anchors were arranged in rectangular formations covering individual product zones, each with four anchors and a nominal coverage radius of approximately 6 m to ensure complete signal coverage and minimize blind spots. The largest zone, Smart Home & IoT, was equipped with eight anchors in an extended rectangular layout to cover its larger area. The system operated at a carrier frequency of 6.5 GHz with a bandwidth of 500 MHz and an update rate of 10 Hz. A hybrid wired-wireless clock synchronization mechanism was employed to maintain temporal consistency, and periodic field calibration was conducted to ensure geometric accuracy. The average positioning error achieved was 0.34 ± 0.20 m under line-of-sight (LOS) and 0.65 ± 0.38 m under non-line-of-sight (NLOS) conditions, with a 95th-percentile error of 1.40 m and a correct zoning rate of 92%. After Kalman filtering and NLOS correction, the cumulative positioning error decreased by approximately 40%, achieving sub-meter accuracy suitable for proximity-based behavioral analysis.

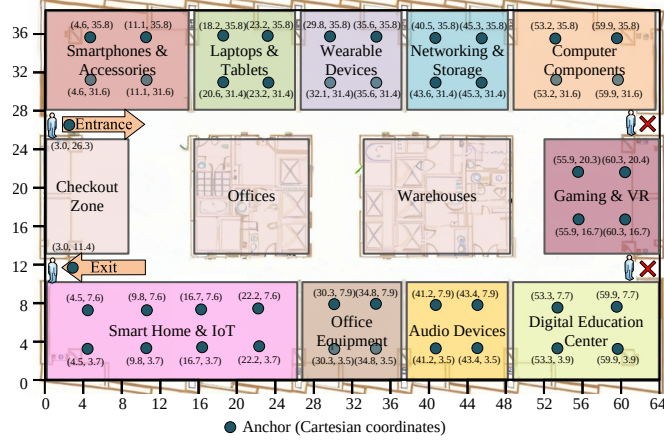


Figure 1: Spatial distribution of UWB anchors within the original store layout.



(a) Zone-Top Anchor. (b) Basket-Bottom Tag. (c) Cart-Bottom Tag.

Figure 2: Experimental Equipment.

To quantify the potential bias, we conducted a data-driven analysis based on the collected UWB trajectories:

Coverage estimation: By comparing the total number of observed trajectories with store entry counts recorded by the point-of-sale and entrance systems, we found that over 90% of shoppers were associated with a tagged

cart or basket. This indicates that only a small fraction (approximately 10%) of shoppers without carts or baskets were potentially unobserved.

Group shopping behavior: Trajectories were analyzed for spatial overlap, revealing that approximately 12% of visits involved groups, of which about 66% shared a single cart. This corresponds to roughly 8% of individual shoppers potentially underrepresented.

Trajectory sparsity and zone sequence robustness: We performed a Monte Carlo simulation to model the effect of these unobserved shoppers. Synthetic trajectories, generated based on the spatial distribution of observed paths, were added to the dataset, and process model metrics (fitness, precision, F-measure) were recomputed. The simulation showed minimal changes (less than 2% deviation in F-measure), indicating that unobserved shoppers have limited impact on the resulting pathway models.

Stationary carts and baskets: Instances where a cart or basket remained in a zone while the associated shopper moved independently were infrequent (less than 3% of total trajectories). Frequency-based thresholds during process mining effectively mitigated any spurious jumps in zone sequences.

These data-driven analyses demonstrate that coverage bias due to missing tags or shared carts is quantitatively small and does not meaningfully affect the discovered consumer pathways, ensuring the reliability of the process mining results.

To accurately capture consumer shopping behavior in event logs, it is essential to define clear rules for case segmentation and to handle rapid repeated visits to the same zones. These steps ensure that each case represents a coherent shopping session and that artificial inflation of transitions does not distort the discovered sequential patterns.

Case segmentation: When a customer exits and re-enters the store or switches from a basket to a cart, a new case is initiated. In such scenarios, the uniqueness of the UWB tag carried by the shopper cannot be guaranteed. In other words, as soon as the tag on the basket or cart is detected by the exit anchor, the current case is considered completed. If the shopper returns to the store or switches to a different cart, this behavior may appear in the final records as a “jump” from the entrance to a new target zone. However, such occurrences are rare. Moreover, during the pathway discovery phase, low-frequency causal relations arising from these events are filtered out by applying frequency thresholds, effectively mitigating their impact.

Handling rapid re-entries (debounce): Rapid, repeated visits to the

same zone, typically resulting from indecision, product consultation, or minor movements within the zone, are addressed through a time-based debounce rule. Specifically, consecutive dwell records within the same zone that are separated by less than 5 seconds are merged into a single visit. This approach helps prevent the artificial inflation of zone transitions while still capturing the natural, cyclic browsing behavior of consumers. In the final event log, the entry time for the zone is recorded as the time of the first entry, while the exit time is recorded as the time of the last departure, along with the duration of the interval. Furthermore, we consider repeated visits as an indication of purposeful engagement with the zone, and therefore, these visit records should not be discarded, regardless of the dwell time.

Preservation of causal relations: Since the method focuses on sequential patterns of zone visits, these segmentation and debounce rules do not distort the causal ordering. Infrequent or erratic transitions (e.g., repeated short excursions to the entrance or other zones) are treated as noise and filtered during process discovery using frequency thresholds.

Quantitative justification: Analysis of the collected UWB trajectories shows that less than 2% of total transitions are affected by case splits or short cyclic movements. Sensitivity tests confirmed that varying the debounce interval between 3–7 seconds leads to minimal changes (<1% deviation in F-measure) in the discovered pathway models.

These data-driven case segmentation and debounce rules ensure that event logs accurately reflect purposeful consumer behavior. By minimizing artifacts from brief exits, tool switches, or rapid cyclic movements, the approach preserves the integrity and robustness of the discovered process models, supporting reliable insights for store layout optimization.

To determine appropriate dwell time thresholds for each anchor and proximity thresholds for each zone, we adopted a data-driven approach. Before the formal experiment, we conducted a pilot study collecting consumer movement traces across all store zones, including both non-purchasing and purchasing behaviors.

For dwell times, we first applied a logarithmic transformation to reduce skewness and better separate behavioral patterns. We then fitted a Gaussian Mixture Model (GMM) to identify two distinct behavioral components: brief incidental stops (passing by) and purposeful dwellings (meaningful interactions). The intersection point of the posterior probabilities of the two components was used to determine the zone-level dwell time threshold, which resulted in a uniform threshold of 20 seconds across all zones. This thresh-

old maximizes the separation between incidental and meaningful behaviors, ensuring that only purpose-driven visits are captured.

Importantly, the 20-second threshold applies consistently across zones, including larger areas. Generally, larger zones tend to contain larger products, such as smart refrigerators and smart TVs in the Smart Home & IoT zone. Consumers can form an initial understanding of these products from a distance, meaning that they do not need to engage with them up close for extended periods. As a result, the 20-second threshold remains effective in identifying meaningful visits in both large and small zones.

Proximity thresholds for each zone were set according to zone area, anchor layout, and spatial coverage. Smaller zones employed shorter distance thresholds, while larger zones, such as Smart Home & IoT, used longer thresholds to reflect their spatial extent and maintain complete coverage. This ensures that each zone’s defined area accurately represents the consumer’s physical presence.

To validate the thresholds, we performed sensitivity analyses varying dwell times (10, 15, 20, 25, 30 s) and proximity thresholds (± 0.3 m around each zone-specific value). The resulting model quality metrics, including fitness, precision, F-measure, generation, and Complexity, are reported in Table 1. Thresholds below 20 s increased false positives by including incidental stops, reducing Precision, while thresholds above 20 s omitted genuine visits, reducing Completeness. Similarly, minor changes in proximity thresholds had only a marginal effect on model metrics, demonstrating robustness. The 20-second dwell time and zone-specific proximity thresholds strike a balance between completeness and accuracy, ensuring that the resulting event logs faithfully represent purposeful consumer behavior.

Consequently, when uploading consumer behavior data to the information system, we calculated the dwell time for each zone based on the corresponding entry and exit timestamps. Zone visits with a dwell time shorter than 20 seconds were removed from the entire behavioral sequence to eliminate noise caused by transient or unintentional movements. Furthermore, if the record for a given zone was composed of multiple visits separated by less than five seconds, it was retained regardless of the total dwell time, to account for purposeful or continuous engagement with the zone.

As summarized in Table 2, the proximity thresholds for each zone range from approximately 2.1 to 5.7 meters, depending on the zone’s area and the level of potential occlusion. Each zone contains multiple overlapping anchors to ensure that consumer movements within the same zone are recorded as

Table 1: Sensitivity analysis of dwell time and proximity thresholds on model quality metrics.

Dwell Time (s)	Proximity (m)	Fitness	Precision	F-measure	Generation	Complexity
10	Default-0.3	0.920	0.940	0.930	0.990	22.0
10	Default	0.922	0.945	0.933	0.990	21.9
10	Default+0.3	0.923	0.943	0.933	0.990	21.8
15	Default-0.3	0.935	0.960	0.947	0.995	21.5
15	Default	0.938	0.965	0.951	0.996	21.4
15	Default+0.3	0.939	0.963	0.951	0.996	21.3
20	Default-0.3	0.950	0.972	0.961	0.998	21.0
20	Default	0.952	0.978	0.965	0.999	20.9
20	Default+0.3	0.953	0.966	0.959	0.999	20.9
25	Default-0.3	0.945	0.970	0.957	0.997	21.2
25	Default	0.947	0.975	0.961	0.997	21.1
25	Default+0.3	0.948	0.973	0.960	0.997	21.0
30	Default-0.3	0.935	0.960	0.947	0.996	21.5
30	Default	0.937	0.965	0.951	0.996	21.4
30	Default+0.3	0.938	0.963	0.951	0.996	21.3

continuous visits rather than fragmented entries. If the proximity threshold is set too low, parts of the browsing trajectory may be missed, and insufficient overlap between anchors within a zone could lead to frequent false “re-entry” records, both of which degrade the quality of the reconstructed pathway model. Conversely, overly large thresholds may cause detections outside the intended zone to be misclassified as valid visits, introducing irrelevant behaviors and reducing the model’s precision. In this configuration, medium-sized zones such as Wearable Devices and Laptops & Tablets use thresholds around 4.5–4.6 meters, while larger or partially NLOS-prone areas like Smart Home & IoT adopt higher thresholds of approximately 5.7 meters to ensure robust zone identification. Smaller transition zones, such as the Entrance and Exit, maintain tighter thresholds near 2.1 meters for accurate entry/exit detection. These values are set safely above the 95th-percentile positioning error to reduce spatial overlap between adjacent zones while maintaining reliable coverage within each area. For overlapping detections near zone boundaries, if both anchors belong to the same zone, no new record is generated and only the dwell duration is updated; if they correspond to different zones, only the record preceding the overlap is retained. This configuration ensures that UWB proximity data can be accurately transformed into high-quality event

logs, supporting reliable process discovery and consumer pathway modeling.

Table 2: Zone configuration and proximity thresholds.

Zone	Area (m ²)	Anchors (n)	Prox. (m)
Smartphones & Accessories	178.56	4	4.9
Laptops & Tablets	114.62	4	4.5
Wearable Devices	113.23	4	4.6
Networking & Storage	116.12	4	4.6
Computer Components	166.55	4	4.9
Gaming & VR	140.80	4	4.7
Digital Education Center	157.50	4	4.9
Audio Devices	110.63	4	4.5
Office Equipment	110.65	4	4.5
Smart Home & IoT	278.89	8	5.7
Entrance	10.00	1	2.1
Exit	10.00	1	2.1

For data collection, the following attributes were recorded: the MAC address of the UWB module, the zone name indicating the consumer’s location, the timestamp when the module entered the zone, and the timestamp when it left (corresponding to the moment when the connection between the anchor and tag was lost). An example of the recorded data is shown in Table 3. The time at which a positioning tag first met the dwell time requirement and exchanged signals with an anchor within a visited zone was defined as the start of the browsing event. The MAC address was then mapped to a Case ID in the event log, ordered by these start timestamps.

Table 3: An Example Event Log Derived from Consumer Movement Data.

MAC Address	Zone	Entry Time	Departure Time
A2:5B:7E:D3:91:4F	<i>A</i>	2025-05-12 09:15:32	2025-05-12 09:20:12
A2:5B:7E:D3:91:4F	<i>B</i>	2025-05-12 09:21:00	2025-05-12 09:28:45
A2:5B:7E:D3:91:4F	<i>E</i>	2025-05-12 09:29:10	2025-05-12 09:35:22
A2:5B:7E:D3:91:4F	<i>G</i>	2025-05-12 09:36:05	2025-05-12 09:42:50
A2:5B:7E:D3:91:4F	<i>E</i>	2025-05-12 09:43:20	2025-05-12 09:50:33

Legend: - *A*: Smartphones & Accessories; *B*: Laptops & Tablets;
E: Computer Components; *G*: Digital Education Center.

2 The ablation experiment of IMopt

We tested three ablation configurations: repairing loops only, repairing choices only, and repairing both loops and choices simultaneously. The results are presented in Table 4 for the toy log and Table 5 for the real event log.

Table 4: Ablation results on the toy example.

Configuration	Fitness	Precision	F-measure	Generation	Complexity
No repair	0.889	0.272	0.417	0.273	17.434
Repair loops only	0.810	0.713	0.756	0.215	7.400
Repair choices only	0.804	0.620	0.702	0.219	7.350
Repair loops + choices	0.821	0.862	0.841	0.222	7.200

Table 5: Ablation results on the real dataset.

Configuration	Fitness	Precision	F-measure	Generation	Complexity
No repair	0.997	0.924	0.959	0.898	19.200
Repair loops only	0.955	0.965	0.960	0.998	20.950
Repair choices only	0.954	0.963	0.958	0.997	21.130
Repair loops + choices	0.952	0.978	0.965	0.999	20.864

Partial improvements from single repairs. On both the toy and real datasets, repairing either loops or choices alone led to a noticeable improvement in precision. Loop repair primarily eliminates redundant self-transitions and misaligned revisits, which are common in retail environments where consumers recheck previously viewed zones. Choice repair targets the most complex choice structures in the process model, specifically those containing the largest number of nodes, in order to constrain overgeneralized branching patterns arising from infrequent or incidental transitions between unrelated zones. For instance, in the real dataset, loop repair increased precision from 0.924 to 0.965, while choice repair achieved 0.963, confirming that each substructure correction effectively reduces distinct types of modeling noise.

Interaction effects in real consumer data. The real event log exhibits a higher degree of structural entanglement, where loops (revisits) often coexist with choices (alternative zone transitions). Many shopping behaviors—such as comparing products in multiple aisles or revisiting high-interest

areas—produce nested loop-choice combinations. Correcting only one type of structure therefore leaves residual inconsistencies in causal ordering, leading to incomplete behavioral representation. This explains why single repairs yield moderate F-measure gains but cannot fully capture the navigation logic of actual shoppers.

Complementarity and superiority of combined repair. When both loops and choices are repaired simultaneously, the improvements become complementary. Loop repair suppresses false repetitions, while choice repair eliminates false branching, producing concurrent gains in both precision and interpretability. After optimization, the substructure may result in a simplified model, or, due to the enhancement of the model’s interpretability, some components may be added, making the model slightly more complex. However, the overall change remains within an acceptable range based on the results.

Behavioral consistency and model interpretability. Qualitatively, the jointly repaired model exhibits higher behavioral consistency: the dominant structures correspond to realistic shopping cycles (e.g., mobile–accessory comparisons or cross-category browsing). In contrast, single-structure repairs occasionally introduced fragmented or semantically implausible transitions, such as direct links between distant categories. This pattern demonstrates that combined structural correction not only improves quantitative measures but also enhances the interpretive fidelity of the discovered pathways to actual consumer behavior.

The ablation analysis clearly shows that while isolated loop or choice repairs yield localized precision improvements, their interaction is essential to forming globally coherent and behaviorally meaningful process models. The combined repair strategy simultaneously enhances model quality and interpretability, accurately reflecting real-world consumer browsing dynamics in complex retail environments.

3 Model Discovery, Transformation and Evaluation

3.1 Model Discovery and Transformation

Taking an event log as input, we can use the ProM plugin *Alpha Miner*, as shown in Figure 3 to generate a Petri net with the default configuration, as

shown in Figure 4.

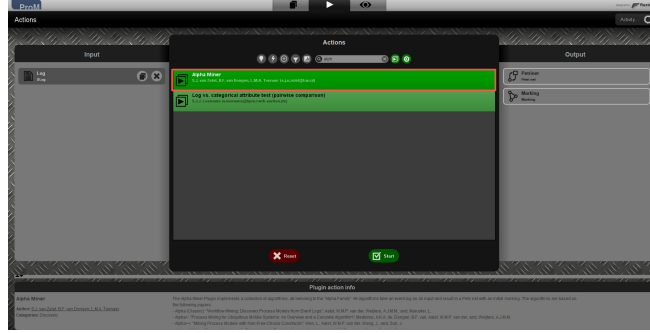


Figure 3: Screenshot of the *Alpha Miner* plugin.

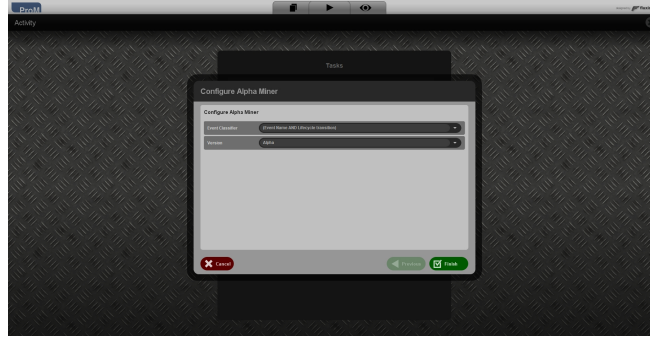


Figure 4: Configuration of the *Alpha Miner* plugin.

Taking an event log as input, we can use the ProM plugin *Mine for a Heuristics Net using Heuristics Miner*, as shown in Figure 5, to generate a Heuristics Net with the default configuration, as shown in Figure 6. Then, we can use the ProM plugin *Convert Heuristics Net into Petri Net*, as shown in Figure 7, to convert the Heuristics Net into a Petri Net. During the conversion, due to the linearization of parallel or long-distance dependencies, some concurrency and loop behavior information may be lost.

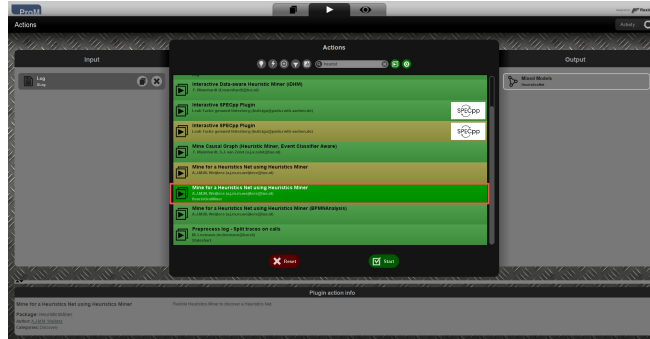


Figure 5: Screenshot of the *Mine for a Heuristics Net using Heuristics Miner* plugin.

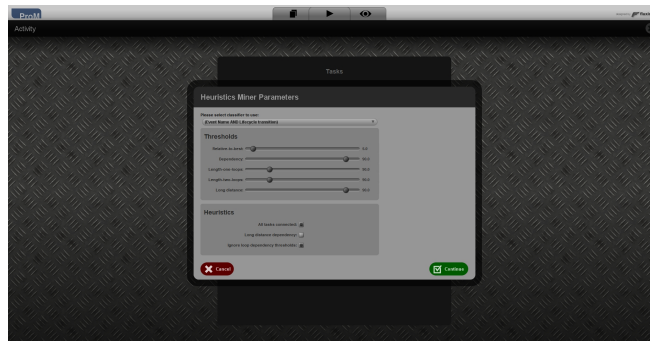


Figure 6: Configuration of the *Mine for a Heuristics Net using Heuristics Miner* plugin.

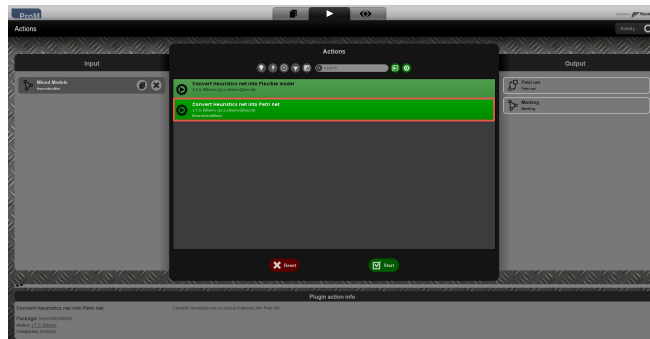


Figure 7: Screenshot of the *Convert Heuristics net into Petri net* plugin.

Taking an event log as input, we can use the ProM plugin *Mine Petri net with inductive Miner*, as shown in Figure 8, to generate a Petri net with the default configuration, as shown in Figure 9.

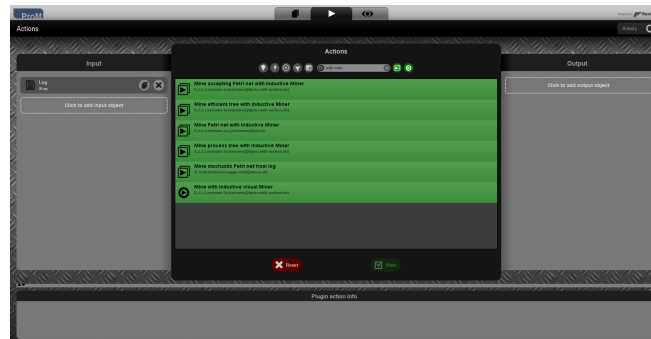


Figure 8: Screenshot of the *Mine Petri net with inductive Miner* plugin.

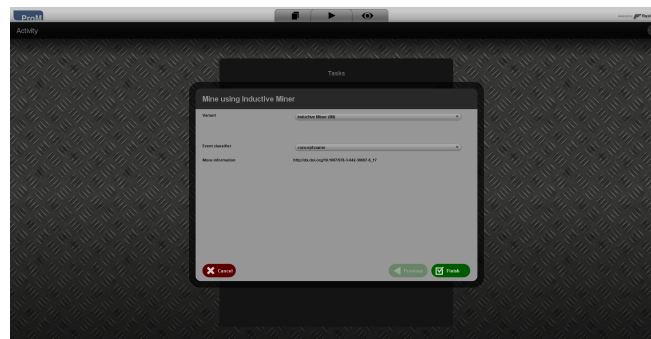


Figure 9: Configuration of the *Mine Petri net with inductive Miner* plugin.

Taking an event log as input, we can use the ProM plugin *Mine Petri net with inductive Miner-infrequent*, as shown in Figure 8, to generate a Petri net with the default configuration, as shown in Figure 10.

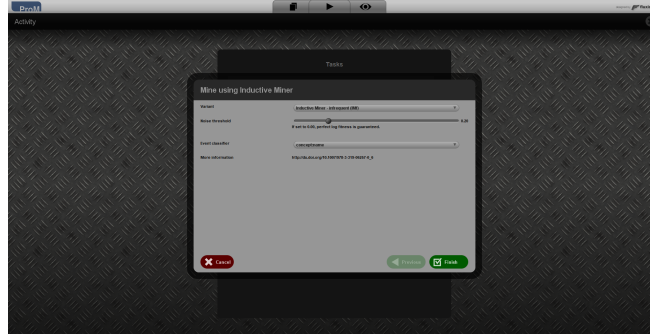


Figure 10: Configuration of the *Mine Petri net with inductive Miner-infrequent* plugin.

Taking an event log as input, we can use the ProM plugin *Mine causal net with Fodina*, as shown in Figure 11, to generate a Causal net with the default configuration, as shown in Figure 12. Then, we can use the ProM plugin *Convert causal Net to BPMN Diagram*, as shown in Figure 13, to convert the Causal net into a BPMN model. Finally, we can use the ProM plugin *Convert BPMN to Petrinet*, as shown in Figure 17, to convert the BPMN model into a Petri Net. Because Causal Nets do not explicitly represent synchronization, certain concurrent or skippable behaviors may be semantically compressed during this process.

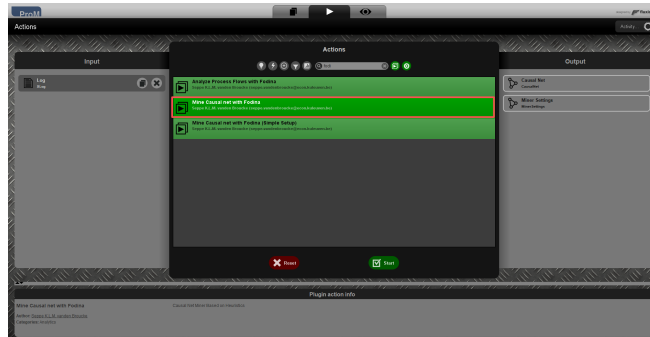


Figure 11: Screenshot of the *Mine causal net with Fodina* plugin.



Figure 12: Configuration of the *Mine causal net with Fodina* plugin.

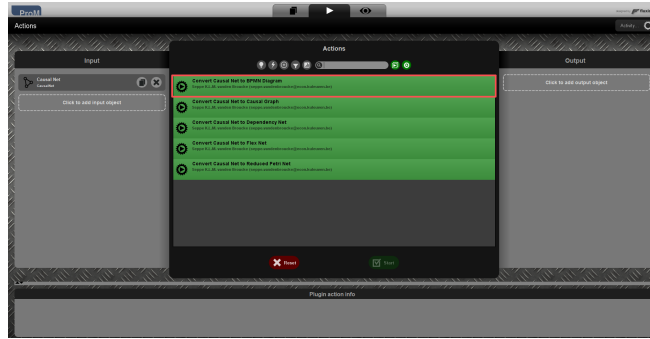


Figure 13: Screenshot of the *Convert causal Net to BPMN Diagram* plugin.

Taking an event log as input, we can use the ProM plugin *Discover BPMN model with Split Miner*, as shown in Figure 14, to generate a BPMN model with the default configuration, as shown in Figure 15 and Figure 16. Then, we can use the ProM plugin *Convert BPMN to Petrinet*, as shown in Figure 17, to convert the BPMN model into a Petri Net. The conversion process will introduce invisible transitions to ensure the model's soundness and reachability.



Figure 14: Screenshot of the *Discover BPMN model with Split Miner* plugin.

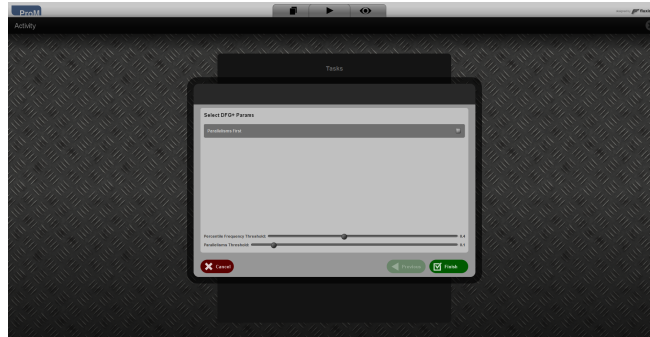


Figure 15: Configuration 1 of the *Discover BPMN model with Split Miner* plugin.

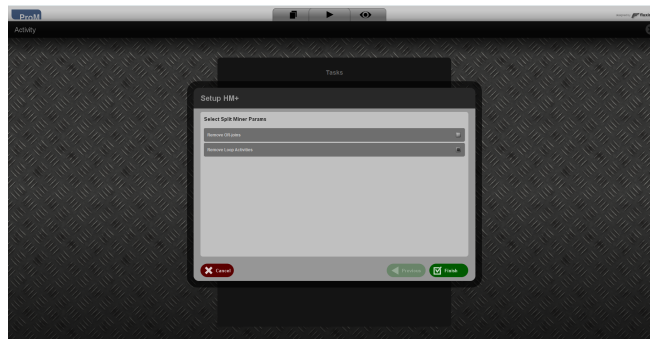


Figure 16: Configuration 2 of the *Discover BPMN model with Split Miner* plugin.

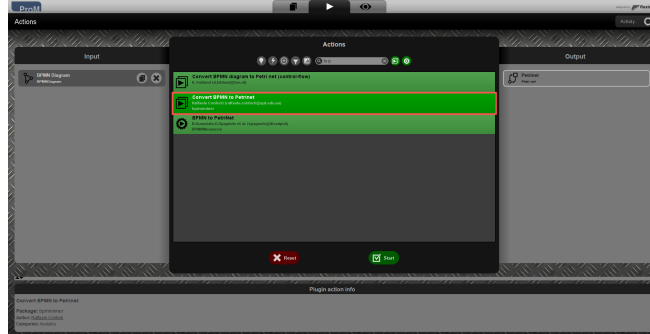


Figure 17: Screenshot of the *Convert BPMN to Petrinet* plugin.

Figure 18 presents the Petri net structures of the seven models discovered using the aforementioned methods.

3.2 Model Evaluation

For measuring the model fitness, we can use the *Replay a Log on Petri Net for Conformance Analysis* plugin, as shown in Figure 19, with the default configuration, as shown in Figure 20, Figure 21 and Figure 22. This plugin takes the event log and Petri net as inputs and generates a panel that displays the model quality.

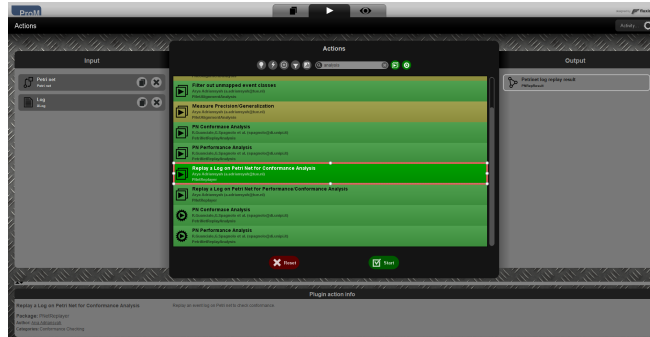
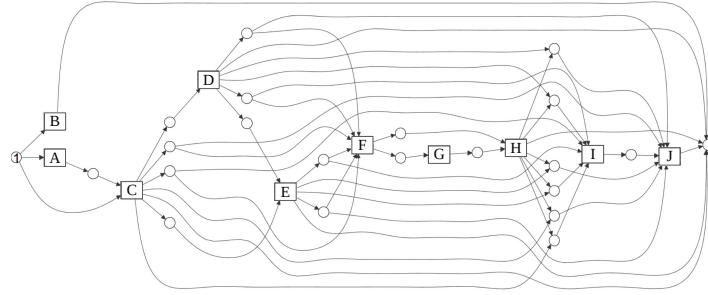
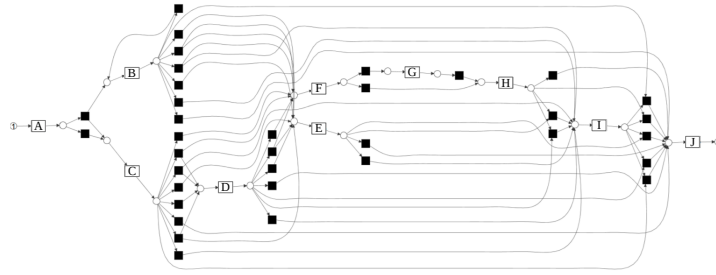


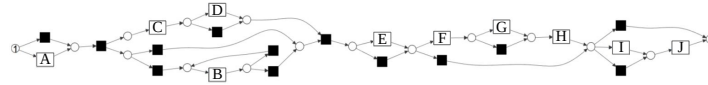
Figure 19: Screenshot of the *Replay a Log on Petri Net for Conformance Analysis* plugin interface.



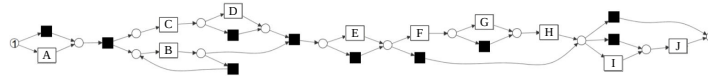
(a) Petri Net Discovered by AM.



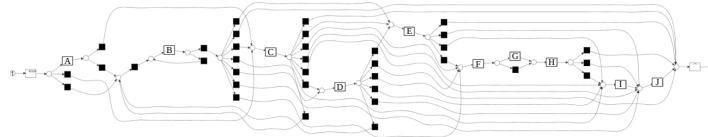
(b) Petri Net Discovered by HM.



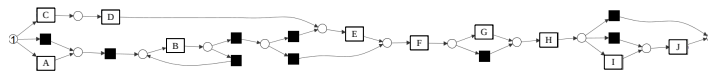
(c) Petri Net Discovered by IM.



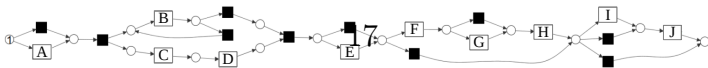
(d) Petri Net Discovered by IMi.



(e) Petri Net Discovered by FM.



(f) Petri Net Discovered by SM.



(g) Petri Net Discovered by IMopt.

Figure 18: Differences Between Consumers' Pathways Process Models.

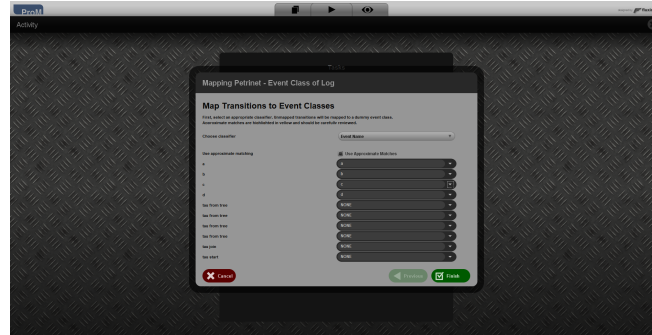


Figure 20: Configuration 1 of the *Replay a Log on Petri Net for Conformance Analysis* plugin.

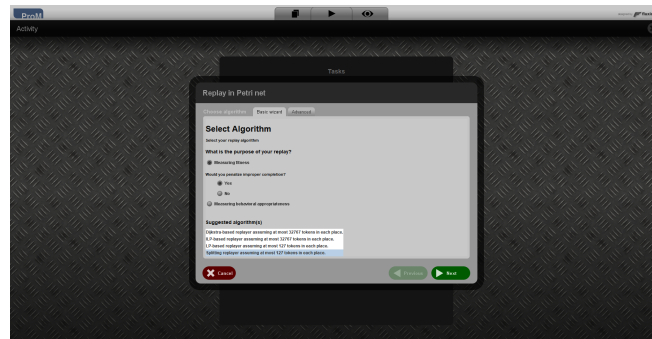


Figure 21: Configuration 2 of the *Replay a Log on Petri Net for Conformance Analysis* plugin.



Figure 22: Configuration 3 of the *Replay a Log on Petri Net for Conformance Analysis* plugin.

For measuring the model precision, we can use the *Check conformance using ETconformance* plugin, as shown in Figure 23, with the default configuration, as shown in Figure 24, Figure 25, Figure 26. This plugin takes the event log and Petri net as inputs and generates a panel that displays the model quality.

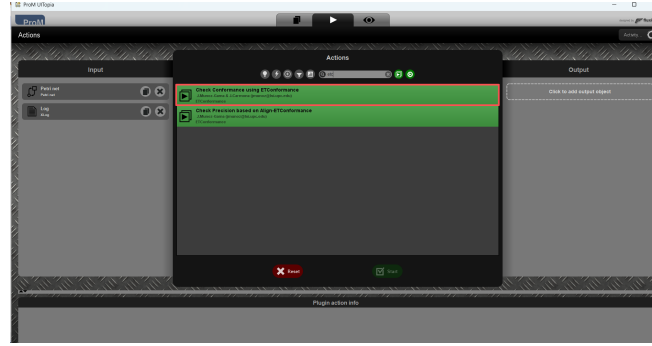


Figure 23: Screenshot of the *Check conformance using ETconformance* plugin interface.

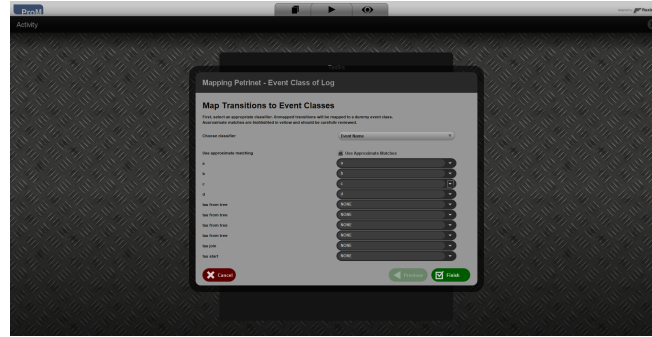


Figure 24: Configuration 1 of the *Check conformance using ETconformance* plugin.

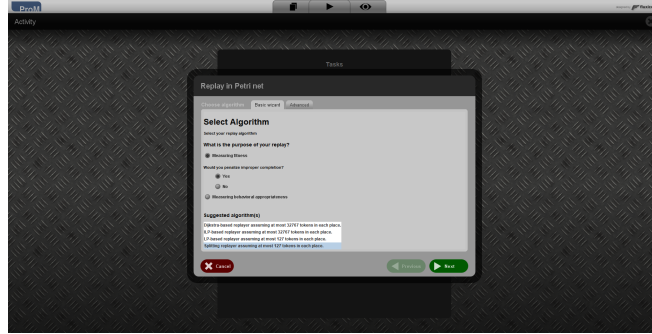


Figure 25: Configuration 2 of the *Check conformance using ETconformance* plugin.



Figure 26: Configuration 3 of the *Check conformance using ETconformance* plugin.

For measuring the model generation, we can use the *Quality Measure of Petri net* plugin, as shown in Figure 27, with the default configuration, as shown in Figure 28, Figure 29, Figure 30, Figure 31. This plugin takes the event log and Petri net as inputs and generates a panel that displays the model quality.

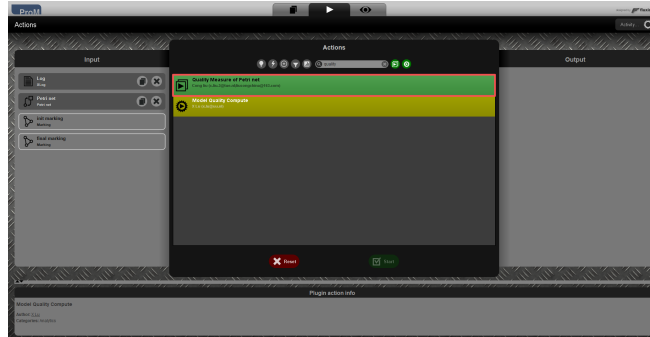


Figure 27: Screenshot of the *Quality Measure of Petri net* plugin interface.

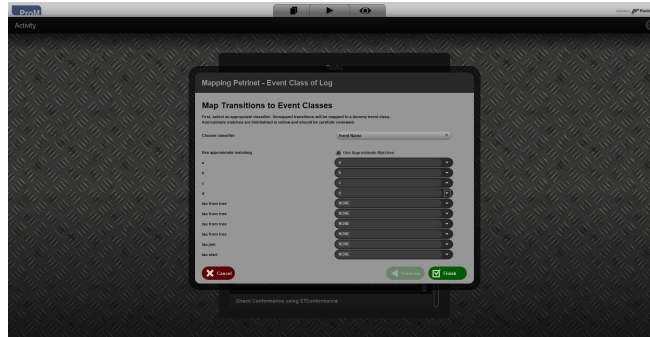


Figure 28: Configuration 1 of the *Quality Measure of Petri net* plugin.

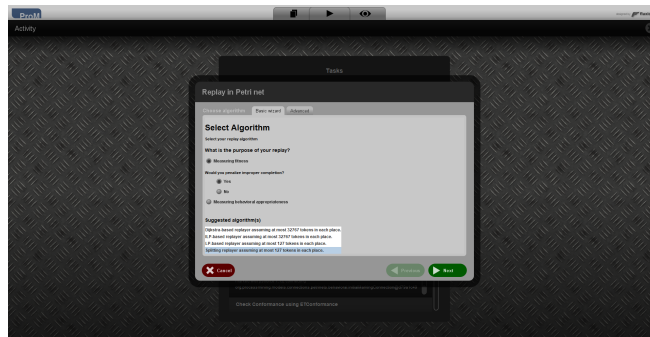


Figure 29: Configuration 2 of the *Quality Measure of Petri net* plugin.



Figure 30: Configuration 3 of the *Quality Measure of Petri net* plugin.

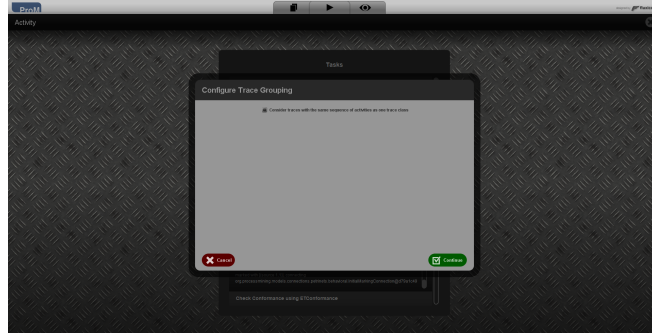


Figure 31: Configuration 4 of the *Quality Measure of Petri net* plugin.

For measuring the model Complexity, we can use the *test Petri Net complexity Plugin* plugin, as shown in Figure 32. This plugin takes the Petri net and its initial marking as inputs and generates a value of model Complexity.

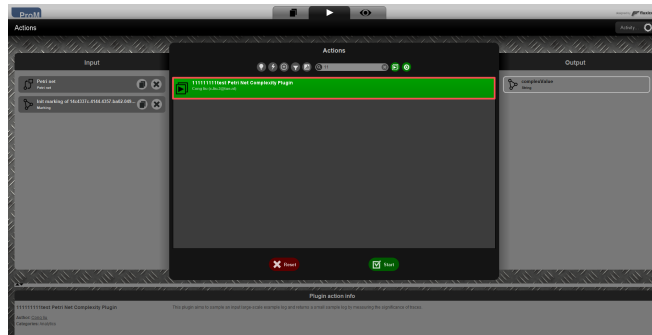


Figure 32: Screenshot of the *test Petri Net complexity Plugin* plugin interface.

For viewing the alignment results, the ProM plugin *Multi-perspective Process Explorer*, as shown in Figure 33, can be used with a Petri net and an event log as inputs. The resulting alignment is illustrated in Figure 34.

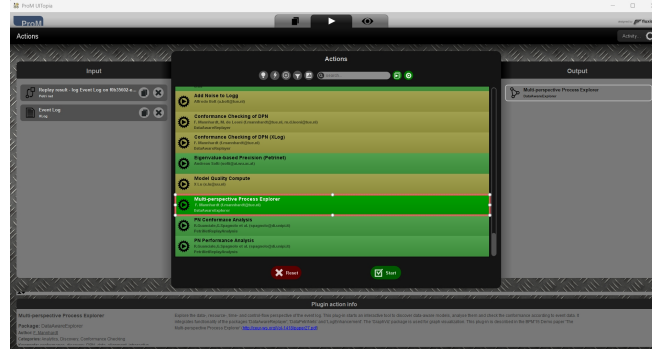


Figure 33: Screenshot of the *Multi-perspective Process Explorer* plugin interface.

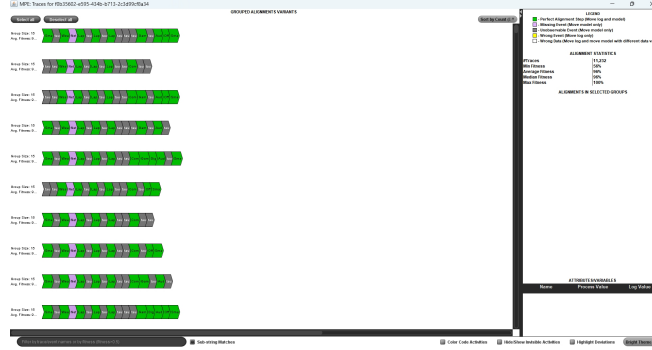


Figure 34: Alignment results produced by the *Multi-perspective Process Explorer* plugin.

4 Data Analysis

Then, we analyzed the specific impact of the adjusted layout on the consumer shopping experience. Figure 35 presents both the number of zones that consumers passed through before reaching their target zone and the average dwell time within each target zone. Proportions were evaluated using the Wilson score interval to construct 95% confidence intervals, with

standard errors computed under binomial assumptions. For instance, among consumers who arrived at the Smartphones & Accessories zone, 34.2% had previously visited only one zone. According to the above statistical configuration, this proportion corresponds to a standard error of 1.5% and a 95% confidence interval of [30.2%, 38.5%], computed using the Wilson method. A two-sample proportion test indicated that this one-hop visiting rate was significantly higher than before the layout adjustment ($z = 2.52$, $p = 0.012$), suggesting that the optimized store configuration effectively encouraged more direct and goal-oriented movement patterns. Similarly, for continuous measures such as dwell time, the mean stay of these one-hop consumers was 64.0 seconds (95% CI: 55.5, 72.5), and no significant difference was observed compared with consumers who entered the zone directly with a purchase intention ($p = 0.28$). The vast majority of consumers passed through no more than three zones before reaching their target zones, indicating that the adjusted layout effectively meets consumers’ browsing preferences and enhances their shopping experience.

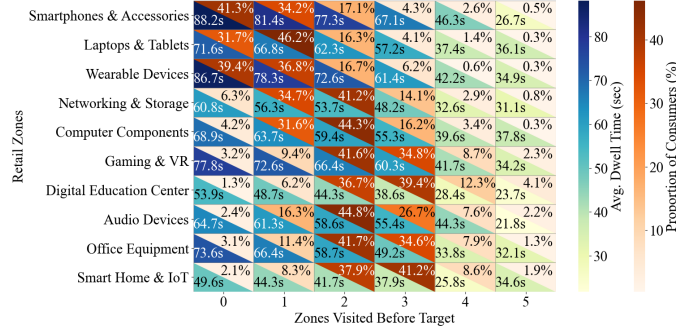


Figure 35: Consumer Dwell Time and Path Proportion Across Retail Zones.

Building on this, we further evaluated the impact of zone-specific factors on consumer experience. Specifically, consumers’ touchpoint interactions with each functional zone were categorized into three types: positive (manifested as empathy, curiosity, imagination, or objective evaluation), neutral, and negative (including dissatisfaction, lack of communication or guidance, or insufficient awareness). During the experimental phase, two independent samples of 500 consumers each were surveyed before and after the layout adjustment using a 1–10 Likert scale (1 representing “extremely dissatisfied” and 10 representing “extremely satisfied”), assessing each zone along two di-

mensions: external factors (zone accessibility, clarity of guidance, in-store atmosphere, and service quality) and internal factors (product price rationality, category richness, brand coverage, and product relevance). For data processing, individual item scores were first standardized (z-score), and principal component analysis (PCA) was applied to extract weights ($KMO = 0.87$, Bartlett’s test $p < 0.001$) to ensure reasonable control of correlations among indicators. Weighted averages were then used to compute composite scores for each dimension, and bootstrapping (10,000 resamples) was employed to obtain 95% confidence intervals for mean differences ($\Delta\%$), improving the robustness of the statistical results. Paired-sample t -tests were conducted to compare pre- and post-adjustment scores, and Cohen’s d was calculated to measure effect size.

As shown in Figure 36, results indicate that internal factor scores remained relatively stable throughout the experiment (score range fluctuation: -2.9% to $+2.9\%$), suggesting that consumers’ evaluations of product content were consistent. In contrast, external factors changed significantly and were concentrated in the high-score range (7–10), with an increase of approximately $+3.95\%$ to $+7.97\%$, while the low-score range (≤ 6) decreased by about 0.28% to 5.69% . Further analysis revealed that positive evaluations of zone accessibility improved most notably, whereas other external factors remained relatively stable. These findings indicate that the layout optimization substantially enhanced consumers’ perception of in-store accessibility, spatial continuity, and guidance clarity, thereby reducing cognitive load during browsing and decision-making. Overall, although internal factors remained stable, optimization of the external spatial structure emerged as a key driver in stimulating shopping motivation and behavioral conversion.

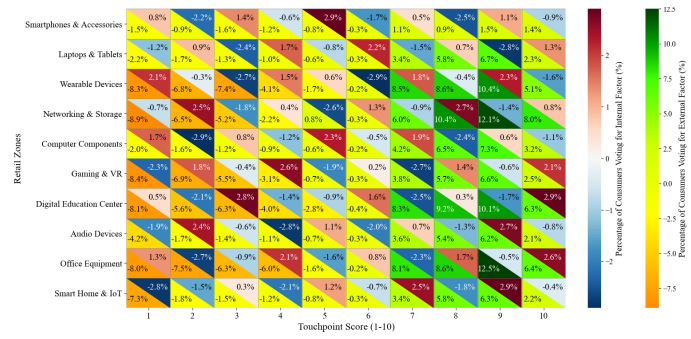


Figure 36: Consumer Touchpoint Feedback on Internal and External Zone Factors.